

DoS & DDoS Attack Simulation

A Local, Capacity-Limited Server Experiment in C++

SUBMITTED BY **Arpita Das**

ROLL NO. 2107021

COURSE CSE 4115

INSTRUCTOR Ms. Dola Das, Assistant Professor, Dept. of CSE, KUET

REPOSITORY <https://github.com/Arpita-Das-2107021/DOS-DDOS-Attack-Simulation>

DATE September 24, 2026

Scope of the Experiment

All traffic stays on 127.0.0.1 (ports 8080/8081). No external host, network, or third party is involved at any point.

1 Objectives

- Understand how DoS and DDoS attacks affect a server's availability and response time.
- Build a small, safe, local lab to simulate normal traffic, DoS traffic, and DDoS traffic.
- Compare server behavior across the three traffic types using real measurements.

2 Introduction

A Denial of Service (DoS) attack floods a server with requests so it cannot serve real users. A Distributed Denial of Service (DDoS) attack does the same thing but from many sources at once, making it harder to stop and more damaging.

This lab simulates both attacks locally on 127.0.0.1, so no real network or external system is affected. A small C++ server acts as the target, and a second C++ program acts as a control panel that generates traffic and shows live results on a web dashboard.

3 Methodology

Two programs were used:

- **server.cpp** – the target server. It can handle 8 requests at the same time. Any request beyond that gets an immediate 503 **Service Unavailable**.
- **client.cpp** – the control panel. It generates traffic and serves a dashboard (`dashboard.html`) to start a phase and watch results live.

The server checks its load before accepting a request:

```
const int CAPACITY = 8;

if (active.fetch_add(1) >= CAPACITY) {
    active--;
    thread(rejectClient, client).detach(); // send 503
    continue;
}
thread(handleClient, client).detach();    // send 200
```

Each request from the client records whether it succeeded, was rejected, or failed:

```
if (response.find("200") != string::npos) gSuccess++;
else if (response.find("503") != string::npos) gRejected++;
else gFailed++;
```

Three traffic phases were tested:

Phase	Workers	Delay between requests
Baseline	1	300 ms
DoS	20	none
DDoS	60	none

Each phase was run from the dashboard for a fixed duration (11 seconds), and the results were recorded automatically.

4 Results

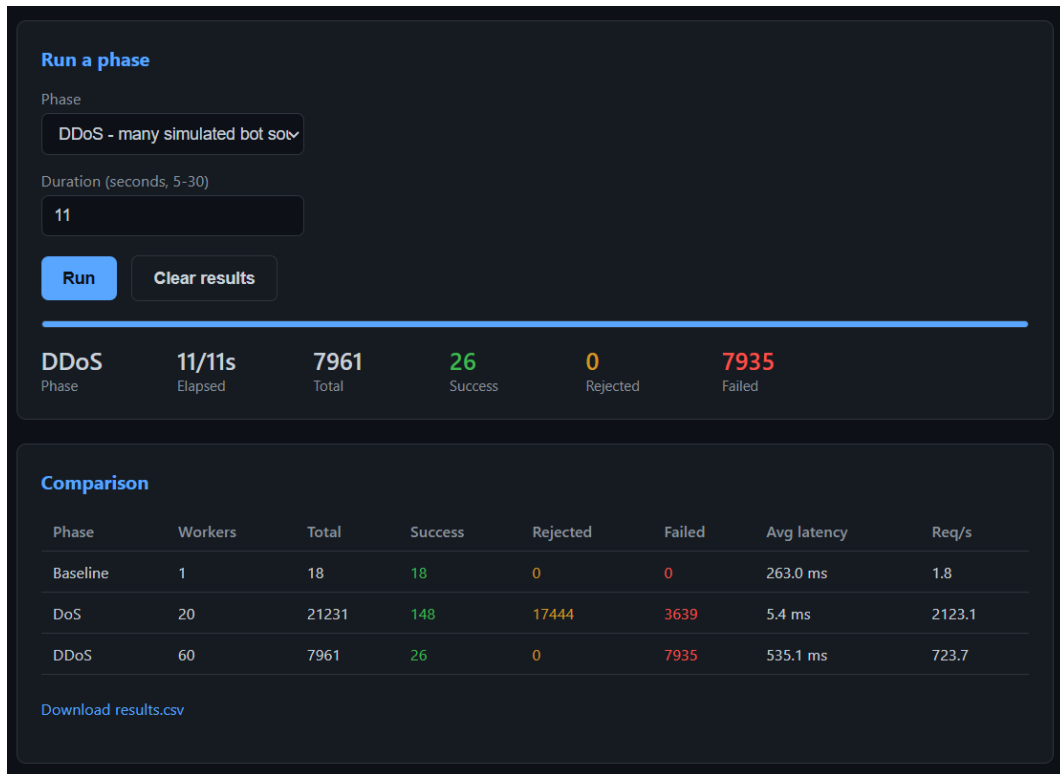


Figure 1: Live dashboard output for the DDoS phase and the accumulated comparison table.

Phase	Workers	Total	Success	Rejected	Failed	Req/s
Baseline	1	18	18	0	0	1.8
DoS	20	21231	148	17444	3639	2123.1
DDoS	60	7961	26	0	7935	723.7

- **Baseline:** every request succeeded. The server handled normal traffic with no problems.
- **DoS:** most requests were cleanly rejected with 503, since the server was over its capacity of 8 but could still keep up and reply.
- **DDoS:** almost every request failed outright (no response at all), and none were cleanly rejected. Throughput also dropped compared to DoS, even with 3x more workers.

5 Discussion

The DoS phase shows the server working as designed: once load goes over capacity, it rejects extra requests with a clean 503 instead of crashing. This is graceful overload handling.

The DDoS phase shows something different and more serious. Since the server creates a new thread for every incoming connection, 60 workers hammering it at once overwhelmed the server's ability to even accept new connections. Requests started timing out or failing before the server could respond, so the "rejected" count stayed at 0 and "failed" shot up instead. Interestingly, total throughput was lower under DDoS than under DoS, showing that more attacking traffic does not always mean more load handled – past a point, the system just collapses.

This matches how real DDoS attacks work: the goal is not just to slow a server down, but to make it completely unreachable, which is more damaging than a server that is simply busy.

6 Conclusion

This lab showed the difference between a server under heavy but manageable load (DoS) and a server pushed past its breaking point (DDoS). A server can be designed to reject excess traffic gracefully, but if the load is too high, it can stop responding altogether. This highlights why real-world systems need defenses like rate limiting, connection limits, and load balancing, not just a capacity check.

References

- [1] Cloudflare, "What is a DDoS Attack?"
<https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>
- [2] OWASP, "Denial of Service"
https://owasp.org/www-community/attacks/Denial_of_Service
- [3] Microsoft, "Winsock Reference"
<https://learn.microsoft.com/en-us/windows/win32/winsock/>